WEALTH PULSE: SYSTEM TEST PLAN

Deliverable ID: WP-TEST-06-0001

Project Title: Wealth Pulse Cross-Asset Portfolio and Educational Suite

Date: June 18, 2026

DOCUMENT METADATA

Document Ownership

Name	Position / Role	Organization	Email
Gage Radtke	Lead Software Engineer / Project Manager	Wilmington University	gradtke001@wilmu.edu
Dr. Charles Bachman	Capstone Advisor / Project Manager Reviewer	Wilmington University	cbachman@wilmu.edu

Revision History

Date	Revision	Summary of Changes	Author
06/18/2026	1.0	Initial Release: Formulated Scenario-Based Coverage Strategy, Validation Matrices, and Acceptance Criteria.	G. Radtke

1. TEST STRATEGY & QUALITY PHILOSOPHY

1.1 Introduction and Role in Design Phase

The *Wealth Pulse* System Test Plan establishes the criteria needed to guarantee technical quality, structural integrity, and architectural compliance for the application. Within the framework of the design phase, the test plan acts as the completion blueprint. If the system successfully satisfies all verification runs defined herein, the Minimum Viable Product (MVP) is formally declared complete and of high quality.

Architectural selections directly shape the testing approach. For example, selecting a decoupled, stateless 3-tier architecture (React frontend meeting a Spring Boot API over a REST boundary) allows the developer to test layers independently. The persistence engine, business calculators, and user views can be validated via targeted test sweeps before assembling them into a unified system.

1.2 The Economics of Code Coverage (The 90/10 Rule)

Test coverage is defined as the total percentage of the code exercised by a verification suite. The total coverage of a suite is the union of individual test cases. While achieving 100% coverage across every internal code path, edge case, and asynchronous branch is an ideal goal, it introduces severe resource constraints in real-world engineering. Achieving 90% coverage is often straightforward, but locking down the final 10% requires a significant amount of development time, which can quickly drain a project's timeline and budget.

Historically, this challenge was demonstrated by IBM in the early 1980s when engineering the Basic Input/Output System (BIOS) to support Microsoft's DOS. Because the BIOS had to be physically burned onto an immutable Read-Only Memory (ROM) chip, patching errors post-release was practically impossible. This required mandatory 100% test coverage. Though the core codebase was small (a few thousand lines), its asynchronous nature required building an entire testing environment just to force the system into specific states before triggering events. The test framework quickly grew much larger than the application itself, introducing the complex challenge of performing quality assurance on the test code. This drove up costs significantly.

To optimize this process, Apple took a more cost-effective approach with the Macintosh by combining a standard ROM with an Electronic Programmable ROM (EPROM). The main codebase lived on the ROM, while any necessary error patches were routed to the rewritable EPROM space.

1.3 Scenario-Based Testing Approach

Taking lessons from these historical paradigms, *Wealth Pulse* avoids chasing expensive 100% path coverage across minor asynchronous threads or internal system branches. Instead, it uses a highly cost-effective Scenario-Based Testing Approach.

By tracing all Typical Scenarios (everyday user workflows) and Atypical Scenarios (edge cases, failure states, and unexpected inputs), the testing suite achieves exceptional software quality within a 14-week timeline. Testing focuses heavily on

high-stakes behaviors—such as the database-backed cache manager and high-precision financial logic—while accepting that minor, internal code branches can be refined during later releases based on alpha user feedback.

2. TESTING ENVIRONMENT & VERIFICATION TOOLS

Verification testing runs completely within the developer's localized engineering stack on Arch Linux, ensuring consistent test conditions:

- Unit Testing Engine: JUnit 5 and AssertJ execute localized math assertions directly within the Code-OSS environment. This process validates that the backend calculation modules return correct values without spinning up the database or network layers.

API Integration Testing: Postman

- runs automated collection scripts against active Spring Boot controllers on port 8080. This step ensures that payload data structures match requirements, headers are processed correctly, and HTTP status responses conform to standard conventions.

Client-Side Interface & Validation UI: Chrome Developer Tools

- tracks the frontend React application. The network console monitors Axios request-response streams, profiles rendering performance, and checks that component-specific CSS Modules scope layouts correctly.

Code Coverage Tracking: JaCoCo (Java Code Coverage)

- is added as a plugin to the Maven pom.xml configuration. It analyzes the backend codebase during compilation runs, providing structural metrics on lines, branches, and instructions exercised by the JUnit suite.

3. TYPICAL SCENARIO TEST SUITE (HAPPY PATHS)

If the running system passes these typical scenarios, it proves the core application infrastructure behaves correctly under standard operating conditions.

Test Case TC-TYP-01: Asset Ledger Data Creation & Database Persistence

Objective:

- Verify that a user can successfully record a physical precious metal asset position via the React interface.

Stimulus Input:

- Navigate to /ledger, open the transaction modal, input Name: "Gold Bullion Maple Leaf", Type: "Precious Metal", Quantity: 5.0000, Purchase Price: 2350.00, and click "Submit."

Expected System Behavior:

- 1. The React app packages the inputs into an AssetDto JSON model and fires an Axios POST request. 2. The Spring Boot backend accepts the payload, matches it to a security token, maps fields to an entity, and writes a row to the PostgreSQL assets table. 3. The database confirms the write operation using high-precision formatting (5.0000 and 2350.0000). 4. The user

interface refreshes automatically, clearing the modal and rendering the new record on the data table.

Test Case TC-TYP-02: Portfolio Dashboard Hydration via Caching Engine

Objective:

- Verify that loading the dashboard extracts asset values through the local cache without exceeding free-tier API rate limits.

Stimulus Input:

- Click on the /dashboard navigation icon when the database contains a market price entry that is 5 minutes old.

Expected System Behavior:

1. The frontend initiates a GET request to /api/assets/summary.
2. AssetService.java intercepts the request and calculates the cache age (Current Time - Cached Timestamp = 300 seconds).
3. Because the age is under the 15-minute expiration window (900 seconds), the server skips outbound web calls to Alpha Vantage or GoldAPI.io.
4. The backend calculates total values using local data, generates a JSON payload, and passes it to the frontend. The dashboard charts render completely within under 40ms.

Test Case TC-TYP-03: Financial Literacy Learning Visualizer Processing

Objective:

- Verify that the interactive options calculator processes option premium values smoothly on the client side.

Stimulus Input:

- Navigate to /learning, locate the Options Visualizer, and drag the underlying stock price slider from $100 up to $115.

Expected System Behavior:

1. The local React component intercepts the change event and updates its internal state.
2. Client-side math modules instantly recalculate option premium changes without making network calls to the backend server.
3. The line graph updates smoothly at 60 FPS, demonstrating clean component isolation and responsive layout behavior.

4. ATYPICAL SCENARIO TEST SUITE (EDGE CASES & FAULT TOLERANCE)

These tests evaluate how safely the architecture catches errors, handles unexpected inputs, and recovers from external integration failures without breaking the user experience.

Test Case TC-ATYP-01: External Third-Party API Outage Recovery

Objective:

- Verify that the application continues to function gracefully if an external data service experiences a network outage.

Stimulus Input:

- Trigger a dashboard page refresh when the database cache is stale (e.g., 2 hours old), while explicitly forcing an outbound connection timeout to GoldAPI.io.

Expected System Behavior:

1. The backend application server detects that the cache has expired and attempts to fetch fresh data via an HTTPS GET request.
2. The connection times out or returns an HTTP 500 error response.
3. The system's integration wrapper catches the exception using an engineered try-catch block, preventing an application crash.
4. The system logs a warning, falls back to the existing database cache entries, calculates the user's balances using those values, and appends a clear notification banner to the frontend UI dashboard.

Test Case TC-ATYP-02: Malformed or Expired Security Token Rejection

Objective:

- Verify that malicious actors or expired client sessions are securely blocked from accessing or altering private asset data tables.

Stimulus Input:

- Submit an Axios GET request to /api/assets/ledger using a forged, unverified security string or an expired JWT.

Expected System Behavior:

1. The Spring Security filter chain catches the incoming request and validates the authorization header signature.
2. The token validation check fails because the signature does not match the server's private secret key.
3. The backend instantly terminates the request, bypassing database access to block data exposure.
4. The system returns an HTTP 401 Unauthorized response status, forcing the client UI to clear localStorage and redirect the browser back to the login screen.

Test Case TC-ATYP-03: Invalid Precision Floats & Malicious Input Sanitation

Objective:

- Verify that the data entry fields reject incorrect string inputs or corrupted values before they reach the calculation engine.

Stimulus Input:

- Attempt to submit an asset transaction form containing a negative quantity value (-2.5) or text characters inside numeric fields ("ten ounces").

Expected System Behavior:

1. Frontend validation blocks the form submission, changing field outlines to red and displaying error text below the invalid items.
2. If a bypassed payload reaches the backend directly, Hibernate validation filters (@Min(0)) catch the input errors.
3. The database transaction is canceled before writing data, preventing data corruption.
4. The API returns an HTTP 400 Bad Request status code, containing a clear error map detailing the validation failures.

5. REVENUE CRITERIA & TRACEABILITY MATRIX

To ensure clear alignment across requirements, design, and testing, this matrix serves as the final sign-off checklist for this capstone project:

Core Functional Goal	System Requirement Source	Verifying Test Case ID	Target Quality Metric Baseline	Status
Asset Balance Persistence	High-precision tracking of physical metals and stocks.	TC-TYP-01, TC-ATYP-03	Exact database storage using DECIMAL(19,4) without rounding errors.	Pass
Dashboard Optimization	Smooth presentation loading that avoids rate-limit locks.	TC-TYP-02, TC-ATYP-01	Dashboard hydration completes in <40ms via local cache checks.	Pass
Financial Education Tool	Interactive client-side options visualizer.	TC-TYP-03	Pure client-side recalculations with zero backend server overhead.	Pass
Security Isolation	Data protection for user accounts and records.	TC-ATYP-02	Irreversible BCrypt password hashing and mandatory JWT verification.	